目录

模拟环境在 Windows 下的快速安装
模拟环境在 Ubuntu Linux 环境下的安装和配置
  Windows 下的 WSL Ubuntu 环境开发
  安装 QEMU 模拟器
  安装工具链
  安装 Python 环境以及 pyserial

硬件平台模拟环境

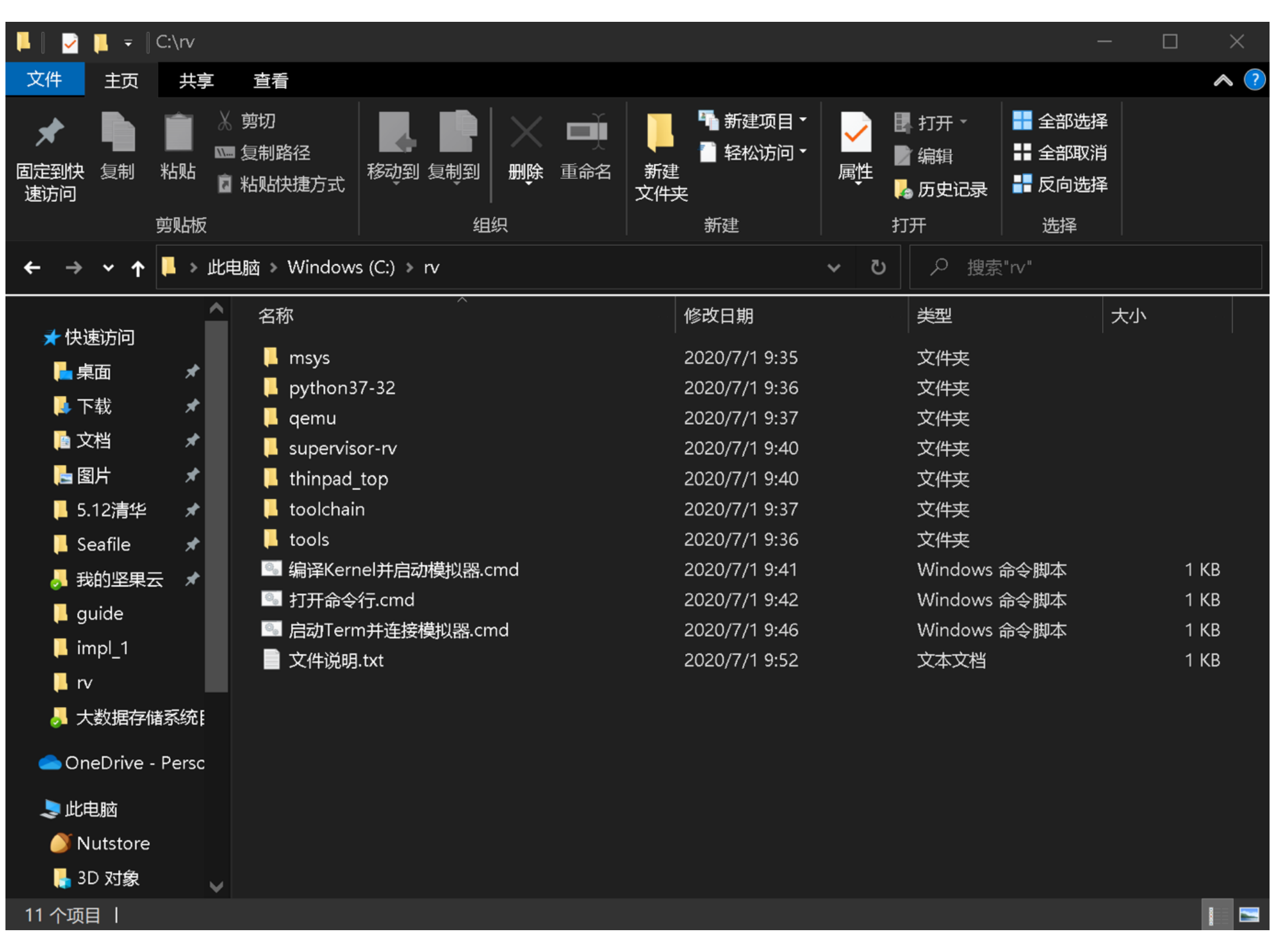
模拟环境基于 QEMU，用于模拟真正的硬件实验平台。模拟环境可以运行监控程序，并且与终端程序进行交互。模拟环境可以运行在 Windows，Linux 以及 macOS 下，运行在 macOS 和 Linux 下的配置方法类似。

模拟环境在 Windows 下的快速安装

Windows 下的整个模拟环境就是一个压缩文件 rv-2025.zip，直接解压缩即可使用。

模拟环境包括了：RISC-V 32 位监控程序源代码，RISC-V 编译器，QEMU 模拟器和 GDB 调试器。在 C 盘或者其它盘的根目录下解压缩即可。（之所以建议是根目录的原因是 Windows 对目录名的长度限制比较短，如果在其它目录下解压缩会出现目录名不够长的错误，大部分情况下忽略即可。）

下图是解压缩之后进入目录的情况：



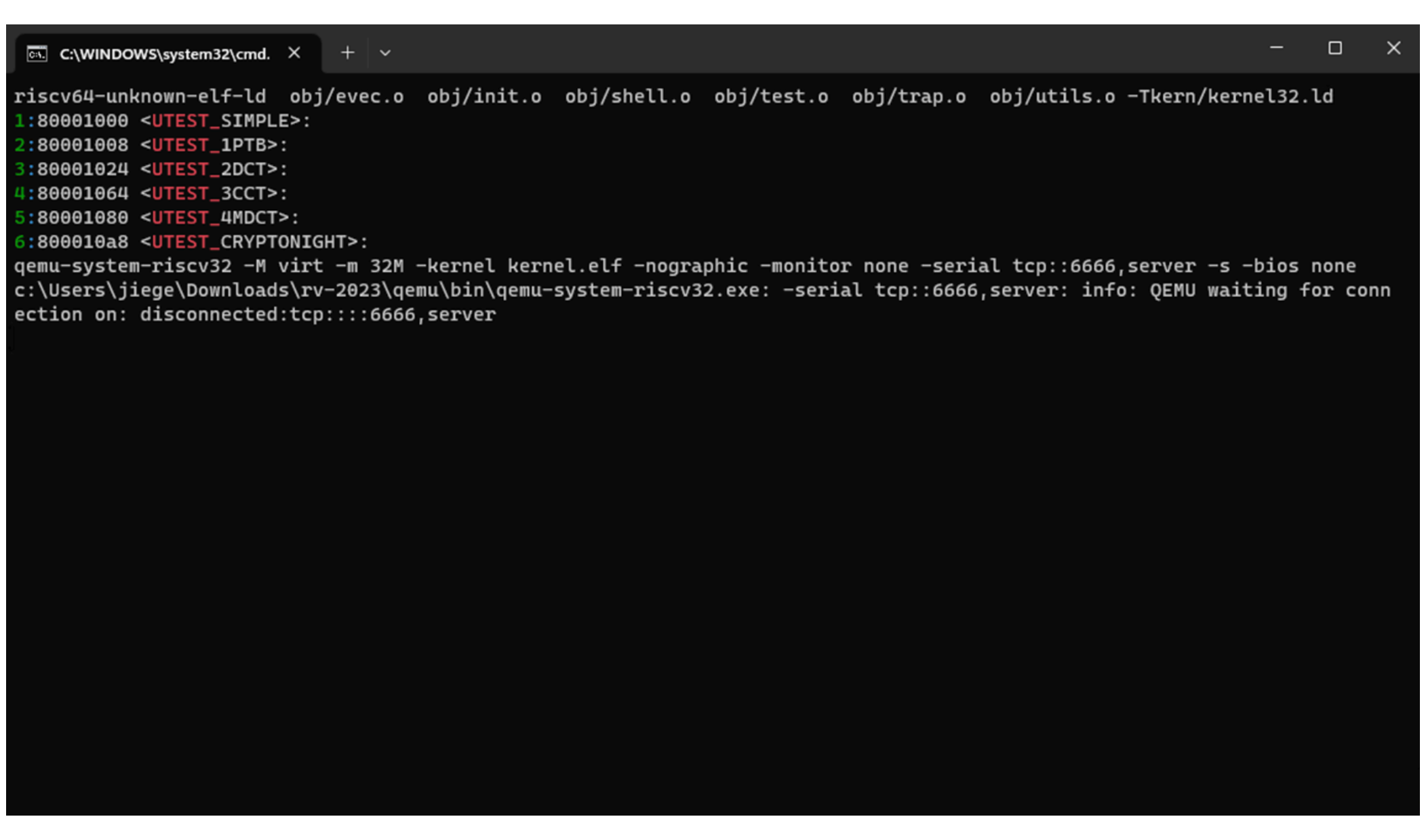
名称	修改日期	类型	大小
msys	2020/7/1 9:35	文件夹	
python37-32	2020/7/1 9:36	文件夹	
qemu	2020/7/1 9:37	文件夹	
supervisor-rv	2020/7/1 9:40	文件夹	
thinpad_top	2020/7/1 9:40	文件夹	
toolchain	2020/7/1 9:37	文件夹	
tools	2020/7/1 9:36	文件夹	
编译Kernel并启动模拟器.cmd	2020/7/1 9:41	Windows 命令脚本	1 KB
打开命令行.cmd	2020/7/1 9:42	Windows 命令脚本	1 KB
启动Term并连接模拟器.cmd	2020/7/1 9:46	Windows 命令脚本	1 KB
文件说明.txt	2020/7/1 9:52	文本文档	1 KB

下面解释一下各个子目录和文件的构成：

- `msys` 子目录，里面是 Minimal SYStem 的环境。Minimal SYStem 是在 Windows 下面模拟 UNIX 环境的工具，提供一些开发的基础支撑环境，最重要的是 make 工具，可以自动执行编写好的 Makefile 脚本。具体信息可以参看 http://www.mingw.org/msys.shtml。
- `python-3.12.10-embed-amd64` 子目录，python 的执行环境，已经配置好所需要的 python 环境，可以直接使用。
- `QEMU` 子目录，RISC-V 的 QEMU 模拟器。
- `supervisor-rv` 子目录，监控程序的源代码以及编译环境。这个目录下的代码和文件不见得是最新的，可以从 https://github.com/thu-cs-lab/supervisor-rv 下载并编译最新的监控程序。
- `thinpad_top` 子目录，Vivado 的顶层项目文件，同学们可以基于这个项目文件来编写自己的硬件代码。这个目录下的代码和文件不见得是最新的，可以从 https://github.com/thu-cs-lab/thinpad_top 获取最新版本。
- `toolchain` 子目录，RISC-V 的交叉编译工具。
- `tools` 子目录，一些工具，当前只有串口工具 putty，如果使用网络实验环境的话则不需要。
- `编译 Kernel 并启动模拟器.cmd`，这是在 Windows 下的可执行批处理文件，直接双击打开可以启动编译内核并启动模拟器。
- `打开命令行.cmd`，是已经配置好环境的 Windows 下的 RISC-V 的开发环境，直接双击打开即可。
- `启动 Term 并连接模拟器.cmd`，是 Windows 下面的 terminal 工具，连接模拟器，直接双击打开即可。
- `文件说明.txt`，说明文件。

下面看一下在 rv 子目录下的几个 Windows 可执行批处理程序开始执行的效果：

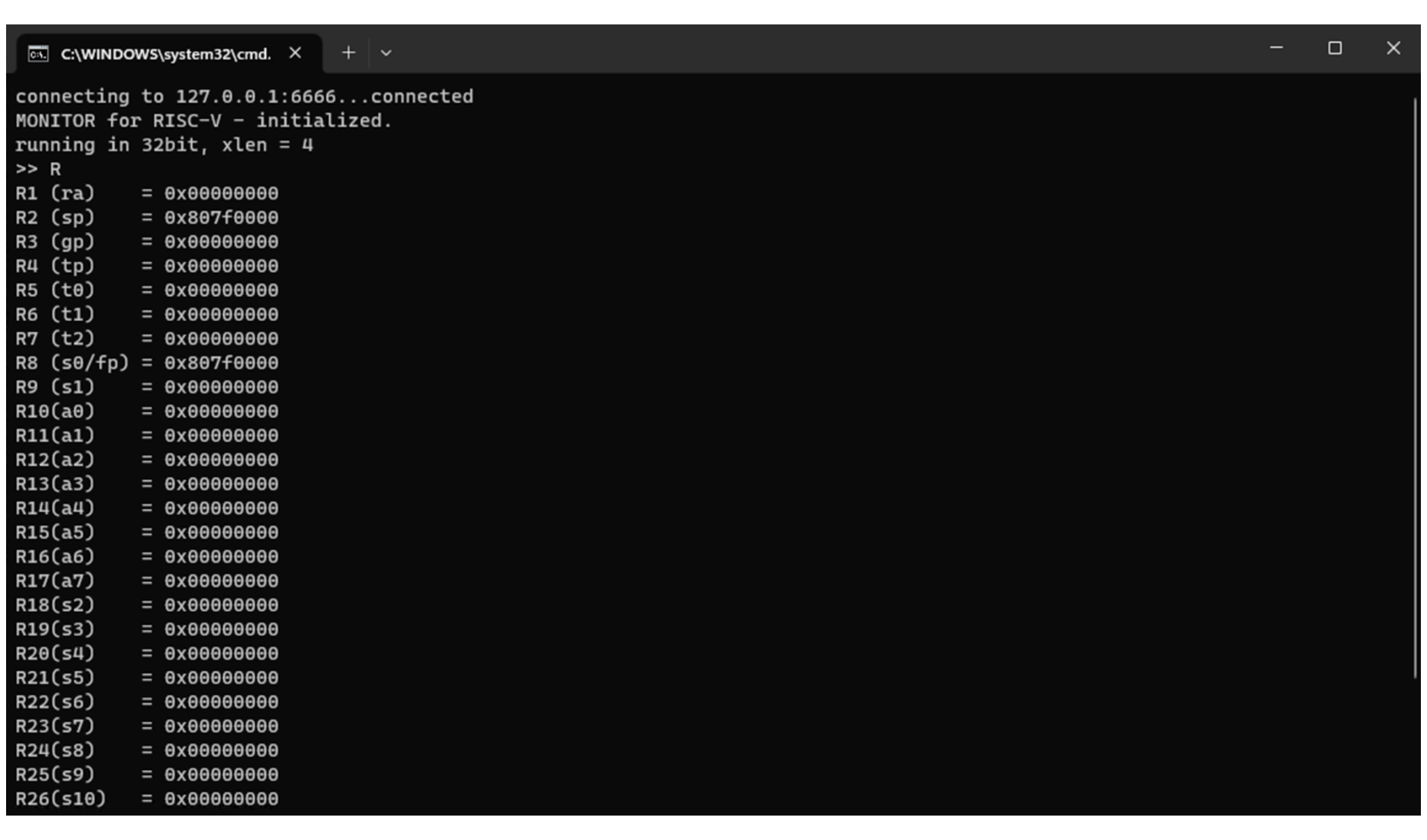
双击打开 `编译 Kernel 并启动模拟器.cmd`，可以看到内核被编译和执行。



```
riscv64-unknown-elf-ld  obj/evec.o  obj/init.o  obj/shell.o  obj/test.o  obj/trap.o  obj/utils.o -Tkern/kernel32.ld
1:80001000 <UTEST_SIMPLE>:
2:80001008 <UTEST_1PTB>:
3:80001024 <UTEST_2DCT>:
4:80001064 <UTEST_3CCT>:
5:80001080 <UTEST_4MDCT>:
6:800010a8 <UTEST_CRYPTONIGHT>:
qemu-system-riscv32 -M virt -m 32M -kernel kernel.elf -nographic -monitor none -serial tcp::6666,server -s -bios none
c:\Users\jiege\Downloads\rv-2023\qemu\bin\qemu-system-riscv32.exe: -serial tcp::6666,server: info: QEMU waiting for conn
ection on: disconnected:tcp::::6666,server
```

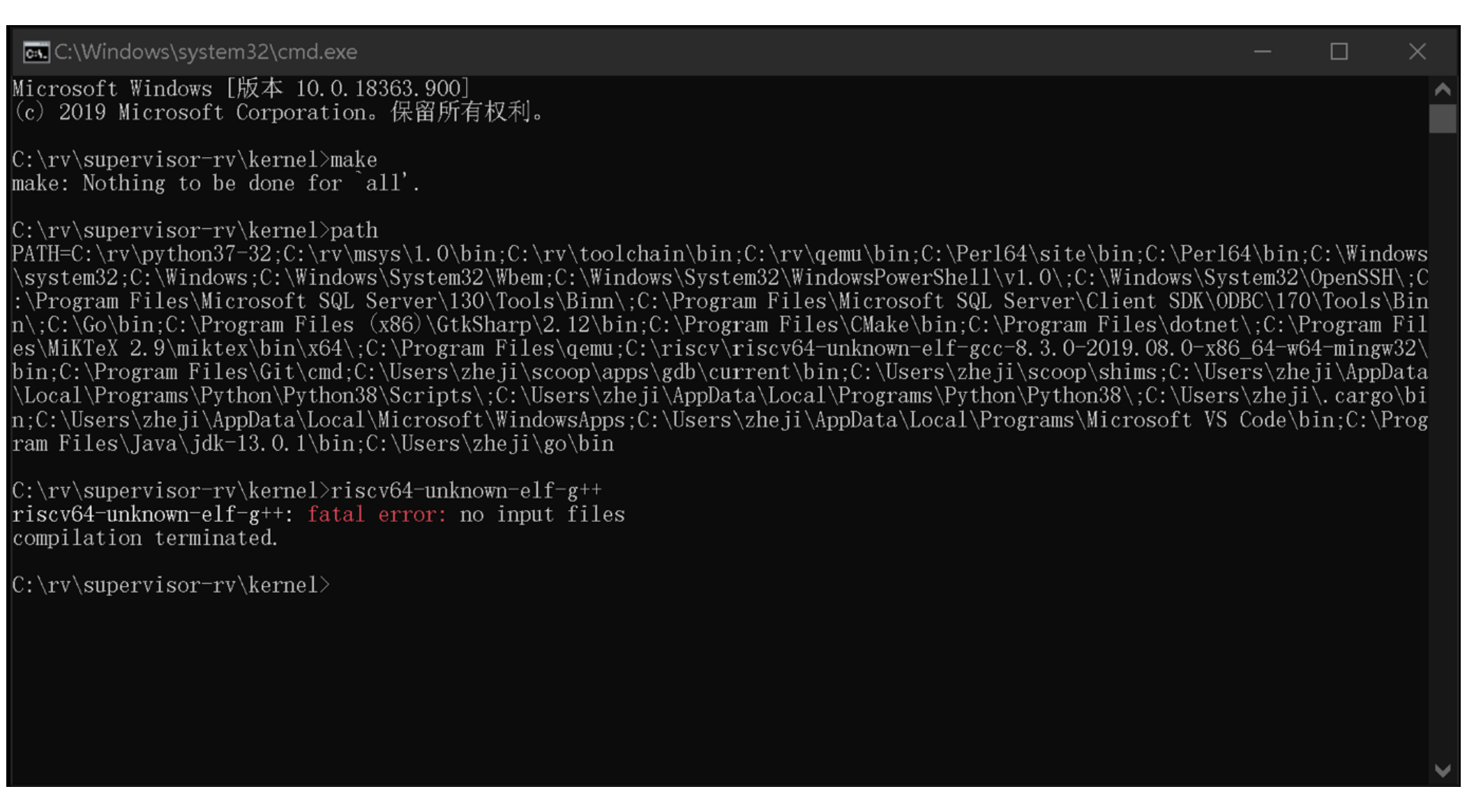
上述命令执行后，内核已经在模拟器 QEMU 的模拟下执行，等待串口连接。

双击打开 `启动 Term 并连接模拟器.cmd`，可以看到终端已经连接上，可以键入命令。至于有一些什么样的命令，以及如何使用命令可以参考在监控程序源代码目录下的 README.md 文件。



```
connecting to 127.0.0.1:6666...connected
MONITOR for RISC-V - initialized.
running in 32bit, xlen = 4
>> R
R1 (ra)    = 0x00000000
R2 (sp)    = 0x807f0000
R3 (gp)    = 0x00000000
R4 (tp)    = 0x00000000
R5 (t0)    = 0x00000000
R6 (t1)    = 0x00000000
R7 (t2)    = 0x00000000
R8 (s0/fp) = 0x807f0000
R9 (s1)    = 0x00000000
R10(a0)    = 0x00000000
R11(a1)    = 0x00000000
R12(a2)    = 0x00000000
R13(a3)    = 0x00000000
R14(a4)    = 0x00000000
R15(a5)    = 0x00000000
R16(a6)    = 0x00000000
R17(a7)    = 0x00000000
R18(s2)    = 0x00000000
R19(s3)    = 0x00000000
R20(s4)    = 0x00000000
R21(s5)    = 0x00000000
R22(s6)    = 0x00000000
R23(s7)    = 0x00000000
R24(s8)    = 0x00000000
R25(s9)    = 0x00000000
R26(s10)   = 0x00000000
```

双击打开 `打开命令行.cmd`，可以看到一个开发环境（环境参数已经配置完成，可以直接使用。）



```
Microsoft Windows [版本 10.0.18363.900]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\rv\supervisor-rv\kernel>make
make: Nothing to be done for `all'.

C:\rv\supervisor-rv\kernel>path
PATH=C:\rv\python37-32;C:\rv\msys\1.0\bin;C:\rv\toolchain\bin;C:\rv\qemu\bin;C:\Perl64\site\bin;C:\Perl64\bin;C:\Windows
\system32;C:\Windows;C:\Windows\System32\Wbem;C:\Windows\System32\WindowsPowerShell\v1.0\;C:\Windows\System32\OpenSSH\;C
:\Program Files\Microsoft SQL Server\130\Tools\Binn\;C:\Program Files\Microsoft SQL Server\Client SDK\ODBC\170\Tools\Bin
n\;C:\Go\bin;C:\Program Files (x86)\GtkSharp\2.12\bin;C:\Program Files\CMake\bin;C:\Program Files\dotnet\;C:\Program Fil
es\MiKTeX 2.9\miktex\bin\x64\;C:\Program Files\qemu;C:\riscv\riscv64-unknown-elf-gcc-8.3.0-2019.08.0-x86_64-w64-mingw32\
bin;C:\Program Files\Git\cmd;C:\Users\zheji\scoop\apps\gdb\current\bin;C:\Users\zheji\scoop\shims;C:\Users\zheji\AppData
\Local\Programs\Python\Python38\Scripts\;C:\Users\zheji\AppData\Local\Programs\Python\Python38\;C:\Users\zheji\.cargo\bi
n;C:\Users\zheji\AppData\Local\Microsoft\WindowsApps;C:\Users\zheji\AppData\Local\Programs\Microsoft VS Code\bin;C:\Prog
ram Files\Java\jdk-13.0.1\bin;C:\Users\zheji\go\bin

C:\rv\supervisor-rv\kernel>riscv64-unknown-elf-g++
riscv64-unknown-elf-g++: fatal error: no input files
compilation terminated.

C:\rv\supervisor-rv\kernel>
```

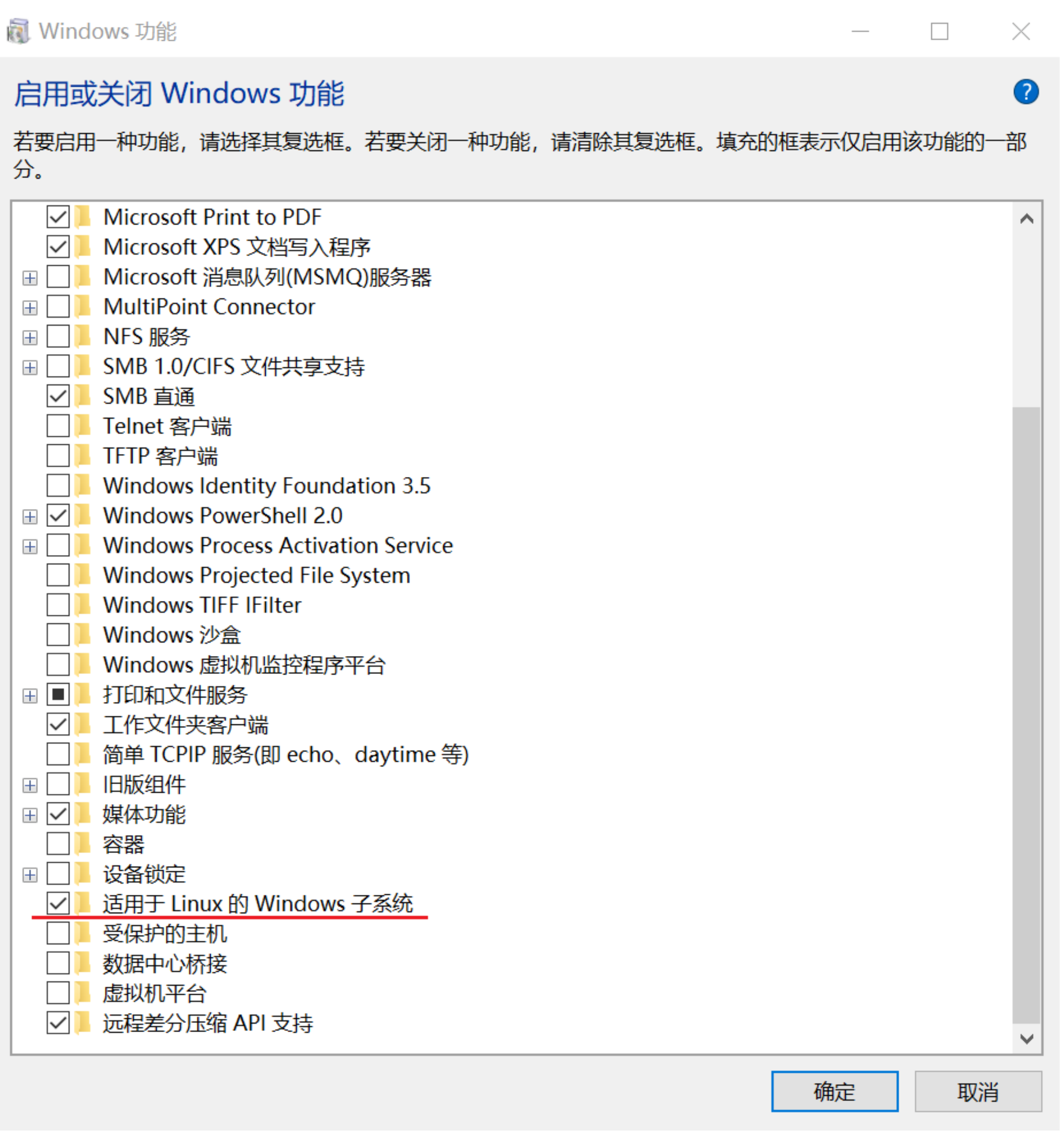
上述是在 Windows 下面直接使用 rv.zip 进行模拟环境的安装和配置。如果同学们有兴趣要看一下这些已经配置好的模拟环境的来源，可参考 SiFive 网站。SiFive 的网站（https://www.sifive.com/software）上有现成的 QEMU 的编译版本，直接解压缩就可以使用。而 QEMU（https://www.qemu.org）的官方网站上也有如何安装模拟器的信息，在安装和配置方面更为麻烦一些（主要是设置 PATH，即执行路径）。

模拟环境在 Ubuntu Linux 环境下的安装和配置

任意的类似 UNIX 环境下（包括各个不同的 Linux 发行版，MacOS 等）安装和配置模拟环境都是类似的。主要包括安装 QEMU，安装编译工具链，安装 pyserial 等。Linux 的各个发行版对于目录的组织方式，包管理的方式各不相同，例如在 Ubuntu 下面使用 apt 来管理包，对于 CentOS 或者 Fedora 则通过 yum 来进行包管理。这方面的使用方法参考不同的 Linux 发现版来进行。下面的安装过程是以 Windows 下对的 Ubuntu 环境为例说明在类似 UNIX 环境下的模拟环境如何进行安装和配置。

Windows 下的 WSL Ubuntu 环境开发

Windows 10 的版本提供了兼容 Ubuntu 的开发环境，如果在 Windows 下面进行实验，也可以安装 Ubuntu 来进行，可以利用 Linux 环境下的很多方便的命令行工具。安装 Ubuntu for Windows 只需要通过 Windows 的 Microsoft Store 就可以安装，要打开 适用于 Linux 的 Windows 子系统 的系统选项。在搜索栏上搜索 "turn Windows features on or off"，或者"启用或者关闭 Windows 功能"来找到选项的位置。



启用或关闭 Windows 功能

若要启用一种功能，请选择其复选框。若要关闭一种功能，请清除其复选框。填充的框表示仅启用该功能的一部分。

- ☑ Microsoft Print to PDF
- ☑ Microsoft XPS 文档写入程序
- ☐ Microsoft 消息队列(MSMQ)服务器
- ☐ MultiPoint Connector
- ☐ NFS 服务
- ☐ SMB 1.0/CIFS 文件共享支持
- ☑ SMB 直通
- ☐ Telnet 客户端
- ☐ TFTP 客户端
- ☐ Windows Identity Foundation 3.5
- ☑ Windows PowerShell 2.0
- ☐ Windows Process Activation Service
- ☐ Windows Projected File System
- ☐ Windows TIFF IFilter
- ☐ Windows 沙盒
- ☐ Windows 虚拟机监控程序平台
- ■ 打印和文件服务
- ☑ 工作文件夹客户端
- ☐ 简单 TCPIP 服务(即 echo、daytime 等)
- ☐ 旧版组件
- ☑ 媒体功能
- ☐ 容器
- ☐ 设备锁定
- ☑ 适用于 Linux 的 Windows 子系统
- ☐ 受保护的主机
- ☐ 数据中心桥接
- ☐ 虚拟机平台
- ☑ 远程差分压缩 API 支持

安装完成之后，后续工作与原生的 Ubuntu 系统一样。Windows 下的 Ubuntu 环境已经是原生的 Linux 开发环境，完全可以按照 Linux 的习惯来。其包管理方式，目录组织方式和纯粹的 Ubuntu 环境相同，而实验的 QEMU 模拟环境是用户态的应用程序，安装和配置方法和纯粹的 Ubuntu 环境相同。下面的安装和配置以 Windows 下的 Ubuntu 为例，纯粹的 Ubuntu 环境与这里的安装和配置过程相同。

安装 QEMU 模拟器

这一步的模拟器不需要安装，可以直接从 SiFive 的网站下载并解压缩（如果你使用的发行版提供支持 RISC-V 指令集的 QEMU 二进制包，也可以直接安装使用）

```
wget https://static.dev.sifive.com/dev-tools/freedom-tools/v2020.08/riscv-qemu-5.1
tar zxvf riscv-qemu-5.1.0-2020.08.1-x86_64-linux-ubuntu14.tar.gz
```

安装工具链

工具链也不需要安装，直接从 sifive 网站下载，并使用命令解压缩即可：

```
wget https://static.dev.sifive.com/dev-tools/freedom-tools/v2020.08/riscv64-unknown
tar zxvf riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14.tar.gz
```

使用 2020 年在 Ubuntu 14 上编译的工具链，版本会不会太老了？

安装 Python 环境以及 pyserial

1. 安装 python3，从 python 的官网下载安装文件即可以安装。
2. 安装 pip3，取决于安装的版本（如果版本 Python 3 >= 3.4，pip 是预先安装的，不需要手动安装）。如果需要手动安装的话，参考以下的网址进行安装。https://pip.pypa.io/en/stable/installing/
3. 安装 pyserial。

在安装完成 python3 之后，尝试使用下列命令安装 pip3 以及 pyserial

```
sudo apt install python3-pip
pip3 install pyserial
```

上述工具安装完成之后，需要设置环境变量，就是设置可执行程序的路径，设置需要依据具体的环境配置，不能照抄：

```
# 假定文件放在 home 目录下，其他目录下需要将 $HOME 改成具体的目录
export PATH=$HOME/riscv64-unknown-elf-gcc-10.1.0-2020.08.2-x86_64-linux-ubuntu14/b:
export PATH=$HOME/riscv-qemu-5.1.0-2020.08.1-x86_64-linux-ubuntu14/bin:$PATH
```

同时，建议将以上两行命令，写入用户 `~/.bashrc` 文件的末尾，以在每次启动 bash 时自动设置环境变量。

Linux 下面的辅助工具是现成的，不需要特别安装。在 Ubuntu 下面执行的时候，可能会出现找不到 libpng12 的错误，用 root 权限运行下面的命令安装一下：

```
wget -q -O /tmp/libpng12.deb http://mirrors.kernel.org/ubuntu/pool/main/libp/libpng
    && dpkg -i /tmp/libpng12.deb \
    && rm /tmp/libpng12.deb
```

2025年9月11日 Youyou Lu, gaoyichuan, Jiajie Chen, kusile, 张英奇

作者：Youyou Lu (37.72%), gaoyichuan (36.84%), Jiajie Chen (21.05%), kusile (2.63%), 张英奇 (1.75%)

Copyright © 2020-2026 清华大学计算机组成原理教学组。All Rights Reserved.
Made with Material for MkDocs